جامعة السلطان مولاي سليمان
Université Sultan Moulay Slimane

المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

Béni Mellal — Maroc

TP 2

Monolithe Modulaire avec Spring Boot

Restructuration et Préparation à la Migration

Réalisé par :

Diallo Thierno Mohammad Lamine

Encadrant :

Pr. BE ELBAGHAZAOUI

Année universitaire : 2025 — 2026

1. Introduction et Objectifs

Ce deuxième travail pratique s'inscrit dans la continuité du TP1 en poussant plus loin la qualité architecturale de l'application Spring Boot. Le monolithe simple développé précédemment est ici restructuré en un monolithe modulaire, où chaque domaine métier (Produit, Client, Commande) est isolé dans son propre module avec des frontières claires et une communication inter-modules définie.

Cette démarche est fondamentale : elle prépare le terrain pour une future migration vers une architecture microservices, tout en maintenant la simplicité d'un déploiement unique. Le patron de conception DTO (Data Transfer Object) est introduit pour découpler la représentation interne des données de leur exposition via l'API REST.

Objectifs du TP

- Restructurer le monolithe du TP1 en modules métier distincts (product, customer, order).
- Introduire le pattern DTO pour séparer les entités JPA des objets exposés à l'API.
- Utiliser MapStruct pour automatiser la conversion Entity ↔ DTO.
- Séparer l'interface de service de son implémentation (ProductService / ProductServiceImpl).
- Implémenter les modules Customer et Order complets avec leurs couches respectives.
- Créer une méthode existsById() dans CustomerService pour vérifier l'existence d'un client.
- Développer un endpoint pour consulter l'historique des commandes d'un client.

Comparaison TP1 vs TP2

Aspect	TP1 — Monolithe Simple	TP2 — Monolithe Modulaire
Couches	Entity, Repository, Service, Controller	+ DTO, Mapper, Service Interface
Exposition API	Entité JPA directement	DTO uniquement
Mapping	Manuel ou absent	MapStruct automatique
Service	Classe concrète unique	Interface + Implémentation séparées
Modules	Product uniquement	Product + Customer + Order
Communication	Accès direct au repository	Via services publics uniquement

2. Architecture Modulaire du Projet

L'application est organisée en trois modules métier autonomes. Chaque module possède exactement la même structure interne en couches, garantissant cohérence et maintenabilité. Le module Order est le seul à dépendre des autres modules, uniquement via leurs interfaces de service publiques — jamais via leurs repositories.

Structure globale du projet

```
com.ecommerce.monolith ├── product │   ├── controller   → ProductController.java │   ├── dto          → ProductDTO.java, CreateProductRequest.java │   ├── mapper       → ProductMapper.java │   ├── model        → Product.java │   ├── repository   → ProductRepository.java │   └── service      → ProductService.java, ProductServiceImpl.java ├── customer │   ├── controller   → CustomerController.java │   ├── dto          → CustomerDTO.java, CreateCustomerRequest.java │   ├── mapper       → CustomerMapper.java │   ├── model        → Customer.java │   ├── repository   → CustomerRepository.java │   └── service      → CustomerService.java, CustomerServiceImpl.java └── order     ├── controller   → OrderController.java     ├── dto          → OrderDTO.java     ├── mapper       → OrderMapper.java     ├── model        → Order.java     ├── repository   → OrderRepository.java     └── service      → OrderService.java, OrderServiceImpl.java
```

Diagramme des dépendances inter-modules

Comme illustré dans le TP, les dépendances entre modules respectent une règle stricte : le module Order consomme uniquement les services publics de Product et Customer, sans jamais accéder à leurs repositories. Product et Customer sont totalement indépendants l'un de l'autre.

Module	Dépendances	Règle
Product Module	Aucune	Totalement autonome
Customer Module	Aucune	Totalement autonome
Order Module	ProductService, CustomerService	Via interfaces uniquement, jamais les repositories

3. Module Product — Refactoring avec DTO et MapStruct

Le module Product du TP1 est entièrement refactorisé. L'entité Product n'est plus exposée directement dans le contrôleur : elle est remplacée par des objets DTO dédiés. MapStruct génère automatiquement le code de conversion entre ces deux représentations à la compilation.

3.1 Vérification préliminaire — GET /api/products

Avant de refactoriser, on vérifie que les données héritées du TP1 sont toujours présentes. Un GET sur /api/products retourne la liste complète des produits avec le code 200 OK. On constate les produits créés lors du TP1 : PC Gamer Zen, Souris Ergonomique, Clavier Mécanique, etc.

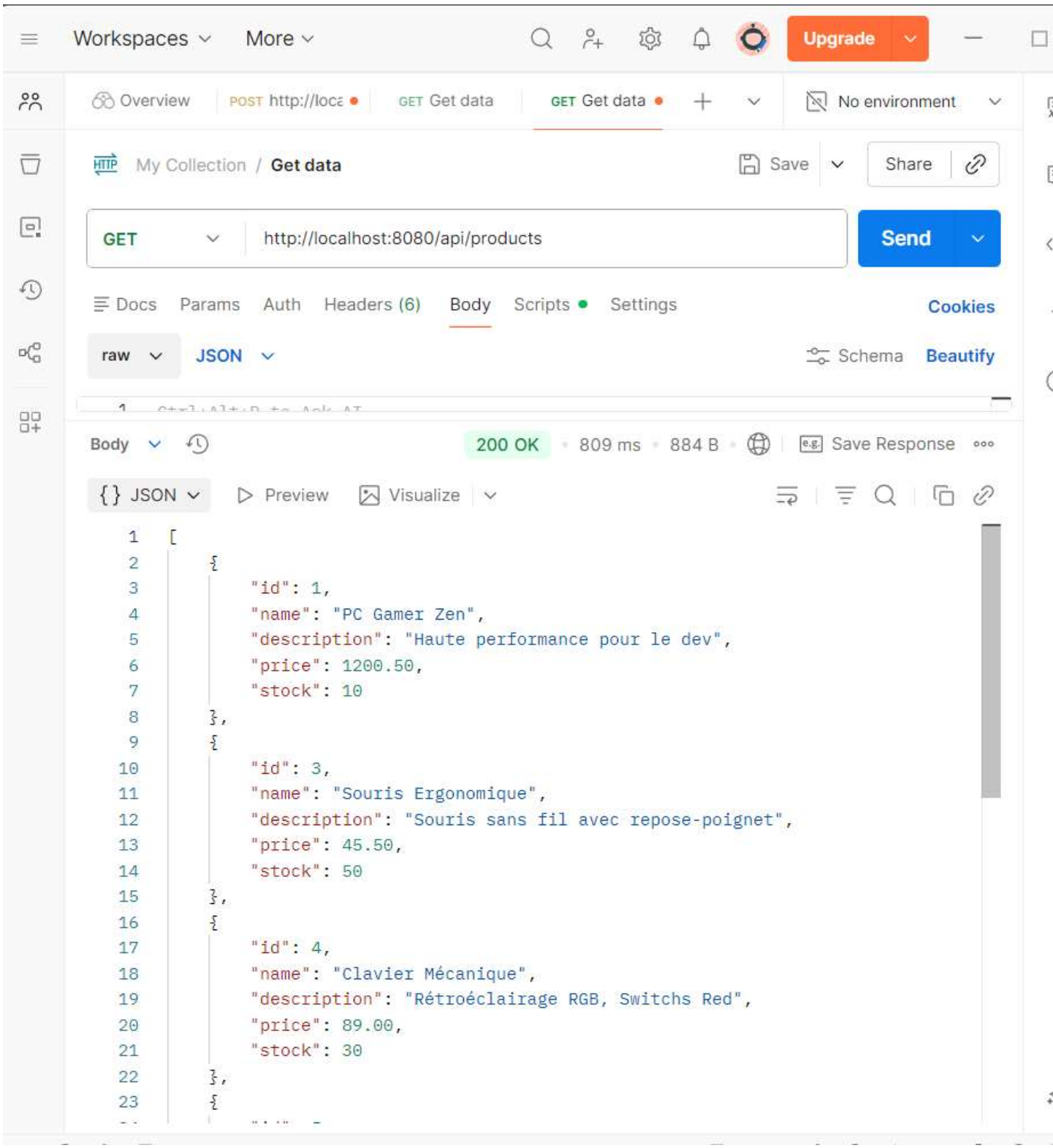

Figure 1 — GET /api/products : liste des produits (200 OK) — héritage du TP1

3.2 POST /api/products — Création via DTO

Après refactoring, la création d'un produit passe par le CreateProductRequest. Ici on crée "Clavier Gamer" (clavier mécanique RGB) à 450,00 avec un stock de 15. La réponse est un ProductDTO (201 Created) — l'entité JPA n'est plus jamais exposée.

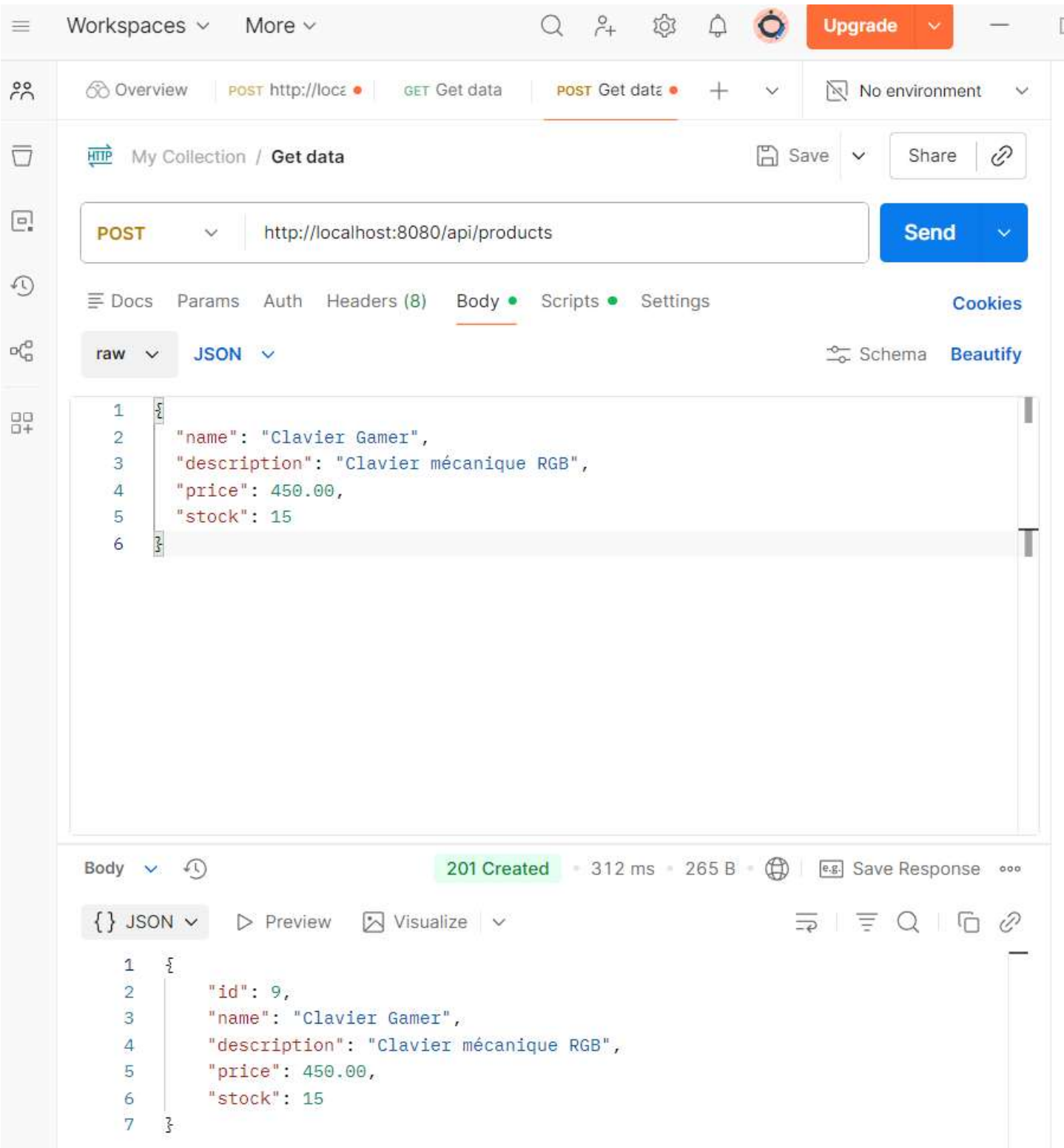

Figure 2 — POST /api/products : création via CreateProductRequest, réponse ProductDTO (201 Created)

3.3 ProductDTO

Le ProductDTO est l'objet renvoyé par l'API. Il contient uniquement les champs nécessaires à l'affichage : id, name, description, price, stock. L'annotation @Data de Lombok génère getters, setters, equals et hashCode automatiquement.

```
erceApplicationTests.java   ProductMapper.java   ProductMapperImpl.java   ProductDTO

1   package com.ecommerce.monolith.product.dto;
2
3   import lombok.Data;
4   import java.math.BigDecimal;
5
6   @Data // Génère automatiquement les Getters et Setters [cite: 37]   25 usages
7   public class ProductDTO {
8       private Long id; //[cite: 39]
9       private String name; //[cite: 40]
10      private String description;// [cite: 41]
11      private BigDecimal price;// [cite: 42]
12      private Integer stock; //[cite: 43]
13  }
```

Figure 3 — ProductDTO.java : objet de transfert exposé par l'API REST

3.4 CreateProductRequest

Le CreateProductRequest est l'objet reçu lors des opérations de création et de mise à jour. Il embarque les validations Jakarta directement : @NotBlank sur le nom, @NotNull et @Positive sur le prix, @PositiveOrZero sur le stock (avec valeur par défaut à 0).

roductMapper.java | ProductMapperImpl.java | ProductDTO.java | CreateProductRequest.java

```java
package com.ecommerce.monolith.product.dto;

import jakarta.validation.constraints.*;
import lombok.Data;
import java.math.BigDecimal;

@Data  15 usages
public class CreateProductRequest {
    @NotBlank(message = "Product name is required") //[cite: 47, 48]
    private String name;

    private String description;// [cite: 49]

    @NotNull(message = "Price is required") //[cite: 52]
    @Positive(message = "Price must be greater than 0")// [cite: 53]
    private BigDecimal price;

    @PositiveOrZero(message = "Stock cannot be negative") //[cite: 54]
    private Integer stock = 0;
}
```

Figure 4 — CreateProductRequest.java : objet de requête avec validations Jakarta

3.5 ProductMapper (MapStruct)

ProductMapper est une interface annotée @Mapper(componentModel = "spring"). MapStruct génère automatiquement l'implémentation ProductMapperImpl à la compilation. Elle fournit quatre méthodes : toDTO(), toDTOList(), toEntity() et updateEntity() — cette dernière met à jour une entité existante sans toucher son identifiant grâce à @Mapping(target = "id", ignore = true).

ProductMapper.java × ProductMapperImpl.java ProductDTO.java CreateProductRec

```java
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.dto.CreateProductRequest;

// Importation de l'entité du modèle [cite: 21]
import com.ecommerce.monolith.product.model.Product;

// Importations MapStruct pour le fonctionnement du Mapper [cite: 55, 56]
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

// Importation utilitaire Java [cite: 60]
import java.util.List;

@Mapper(componentModel = "spring") // [cite: 56]  3 usages  1 implementation
public interface ProductMapper {

    ProductDTO toDTO(Product product); // [cite: 58]  4 usages  1 implementation

    List<ProductDTO> toDTOList(List<Product> products); // [cite: 60]  1 usage  1

    Product toEntity(CreateProductRequest request); // [cite: 61]  1 usage  1 imple

    @Mapping(target = "id", ignore = true) // [cite: 62]  1 usage  1 implementation
    void updateEntity(CreateProductRequest request, @MappingTarget Product proc
}
```

Figure 5 — ProductMapper.java : interface MapStruct avec les 4 méthodes de conversion

3.6 ProductService — Interface et Implémentation

Le service est désormais divisé en deux fichiers. ProductService est l'interface publique du module — elle définit le contrat que respecte l'implémentation. Cette séparation permet de changer l'implémentation sans impacter les consommateurs du service.

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.dto.CreateProductRequest;
import java.util.List;

public interface ProductService {
    // On renvoie une liste de DTOs pour l'affichage
    List<ProductDTO> getAllProducts();

    // On récupère un seul DTO par ID
    ProductDTO getProductById(Long id);

    // On reçoit une requête de création et on renvoie le DTO créé
    ProductDTO createProduct(CreateProductRequest request);

    // Mise à jour via la requête DTO
    ProductDTO updateProduct(Long id, CreateProductRequest request);

    void deleteProduct(Long id);
}
```

Figure 6 — ProductService.java : interface définissant le contrat public du module

ProductServiceImpl implémente l'interface. Elle injecte ProductRepository et ProductMapper via @RequiredArgsConstructor. Chaque méthode utilise le mapper pour convertir les entités en DTO avant de les retourner, garantissant que l'entité JPA ne sort jamais du service.

ProductServiceImpl.java × ProductService.java ProductController.java MonolithEc

```java
14  public class ProductServiceImpl implements ProductService {
26      public ProductDTO getProductById(Long id) {
28              .orElseThrow(() -> new RuntimeException("Produit introuvable"
29          return mapper.toDTO(product);
30      }
31
32      @Override  1 usage
33      public ProductDTO createProduct(CreateProductRequest request) {
34          // On transforme la requête en entité pour la base de données
35          Product product = mapper.toEntity(request);
36          return mapper.toDTO(repository.save(product));
37      }
38
39      @Override  1 usage
40      public ProductDTO updateProduct(Long id, CreateProductRequest request) {
41          Product product = repository.findById(id)
42              .orElseThrow(() -> new RuntimeException("Produit introuvable"
43          // Mise à jour de l'entité existante via le Mapper
44          mapper.updateEntity(request, product);
45          return mapper.toDTO(repository.save(product));
46      }
47
48      @Override  1 usage
49      public void deleteProduct(Long id) {
50          repository.deleteById(id);
51      }
52  }
```

Figure 7 — ProductServiceImpl.java : implémentation avec mapper intégré

3.7 ProductController refactorisé

Le contrôleur travaille exclusivement avec des ProductDTO et des CreateProductRequest. Il n'a plus aucune connaissance de l'entité Product ni du repository. Il délègue toute la logique à l'interface ProductService injectée.

MonolithEcommerceApplication Start Free Trial

ProductServiceImpl.java ProductService.java ProductController.java MonolithEcommer

```java
17  public class ProductController {
21      @GetMapping  new *
22      public List<ProductDTO> getAllProducts() {
23          return productService.getAllProducts();
24      }
25
26      @GetMapping("/{id}")  new *
27      public ResponseEntity<ProductDTO> getProductById(@PathVariable Long id) {
28          return ResponseEntity.ok(productService.getProductById(id));
29      }
30
31      @PostMapping  new *
32      @ResponseStatus(HttpStatus.CREATED)
33      public ProductDTO createProduct(@Valid @RequestBody CreateProductRequest reques
34          return productService.createProduct(request);
35      }
36
37      @PutMapping("/{id}")  new *
38      public ResponseEntity<ProductDTO> updateProduct(
39              @PathVariable Long id,
40              @Valid @RequestBody CreateProductRequest request) {
41          return ResponseEntity.ok(productService.updateProduct(id, request));
42      }
43
44      @DeleteMapping("/{id}")  new *
45      @ResponseStatus(HttpStatus.NO_CONTENT)
46      public void deleteProduct(@PathVariable Long id) {
47          productService.deleteProduct(id);
48      }
49  }
```

Figure 8 — ProductController.java refactorisé : endpoints REST avec DTOs uniquement

4. Module Customer

Le module Customer est implémenté de zéro, en appliquant exactement la même architecture que le module Product refactorisé : entité, repository, DTO, mapper, service interface + implémentation, et contrôleur. Ce module est totalement autonome et n'a aucune dépendance sur les autres modules.

4.1 Entité Customer

La classe Customer est mappée sur la table "customers" via @Entity et @Table. Elle possède un identifiant auto-incrémenté, un prénom (firstName), un nom de famille (lastName), et un email unique non nul. L'annotation @Data de Lombok génère l'ensemble du boilerplate.

```java
package com.ecommerce.monolith.customer.model;

// Ces imports corrigent "Entity", "Id", "GeneratedValue", "GenerationType", "Colu
import jakarta.persistence.*;

// Cet import corrige "Data"
import lombok.Data;

@Entity
@Data
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    private String lastName;

    @Column(unique = true, nullable = false)
    private String email;
}
```

Figure 9 — Customer.java : entité JPA avec contraintes email unique et firstName non nul

4.2 CustomerRepository

CustomerRepository étend JpaRepository<Customer, Long>. Il ajoute une méthode dérivée existsByEmail(String email) — cette méthode répond à l'étape 2 de l'exercice du

TP : permettre de vérifier l'existence d'un client par son email avant de le créer, évitant ainsi les doublons.

Figure 10 — CustomerRepository.java avec existsByEmail() et la structure du module visible dans l'arborescence

4.3 CustomerService — Interface

L'interface CustomerService définit quatre méthodes : createCustomer(), getAllCustomers(), getCustomerById() et existsById(). Cette dernière est l'implémentation de l'exercice 2 du TP : vérifier l'existence d'un client par son ID, ce que le module Order utilisera pour valider les commandes.

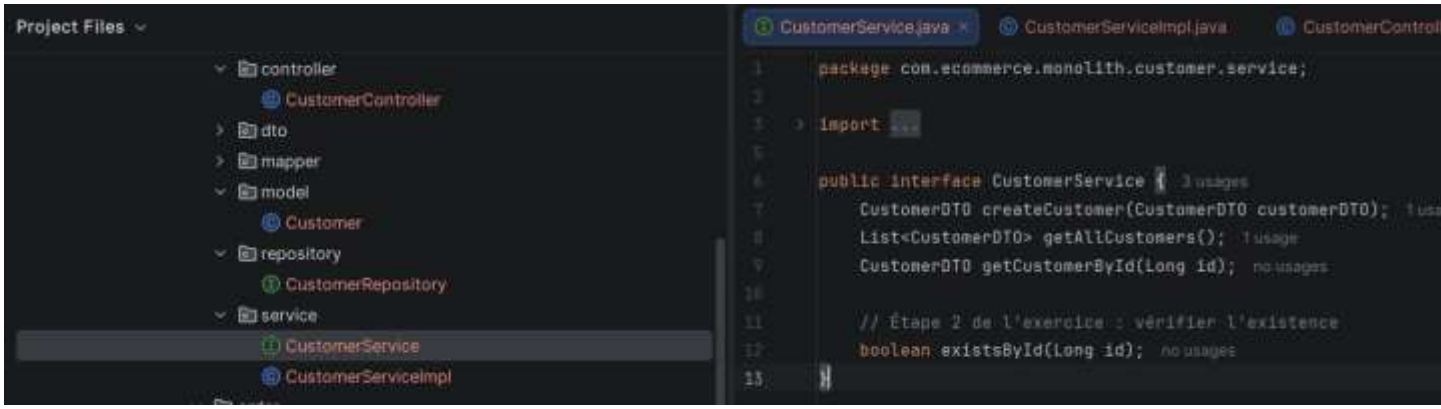

Figure 11 — CustomerService.java : interface avec la méthode existsById() pour l'exercice 2

4.4 CustomerServiceImpl

L'implémentation injecte CustomerRepository et CustomerMapper. La méthode createCustomer() convertit le DTO en entité, le persiste et renvoie le DTO résultant. getAllCustomers() utilise toDTOList(). getCustomerById() charge l'entité et la convertit via toDTO(), lançant une RuntimeException si introuvable. existsById() délègue directement au repository.

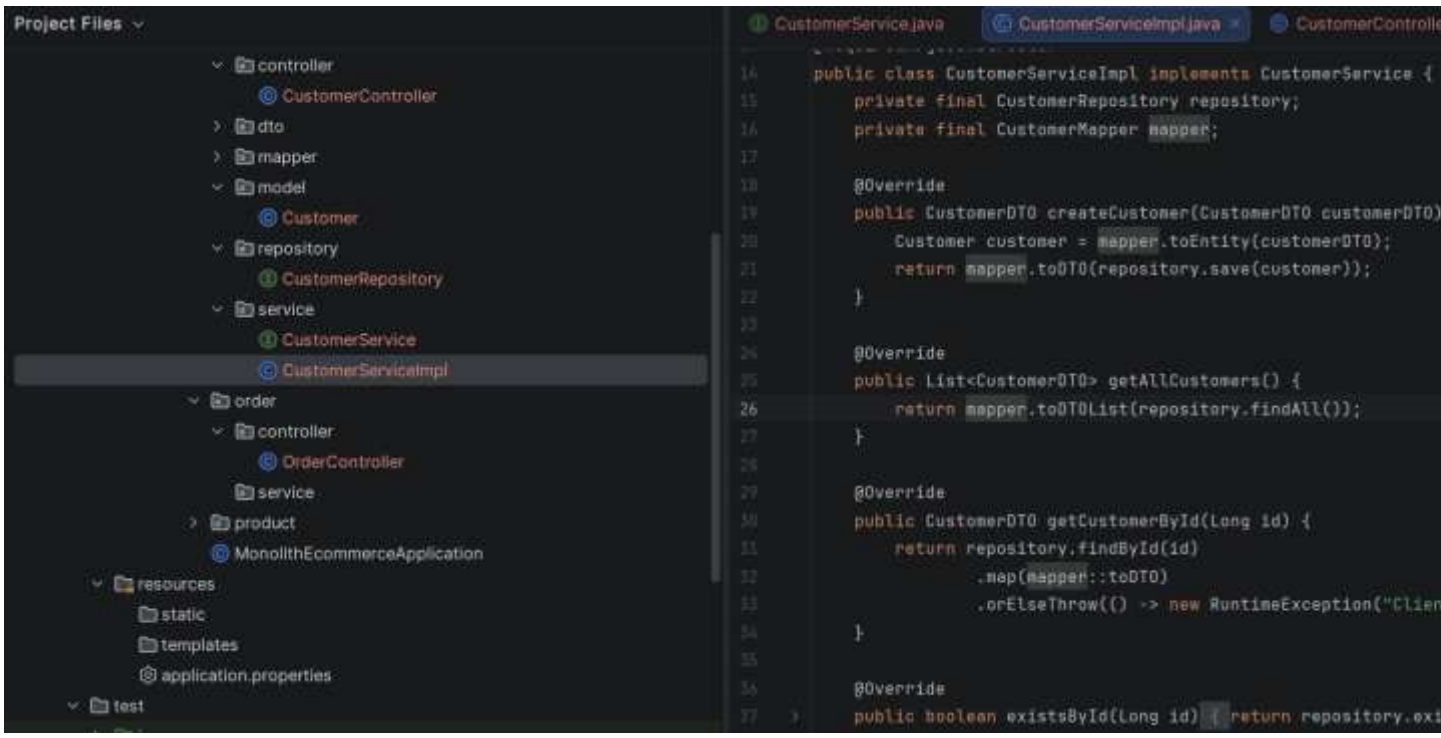

Figure 12 — CustomerServiceImpl.java : implémentation complète avec mapper et gestion d'erreur

4.5 CustomerMapper et DTOs

CustomerMapper est une interface MapStruct @Mapper(componentModel = "spring") qui génère les conversions Customer ↔ CustomerDTO. Il expose toDTO(), toEntity() et toDTOList(). La structure du module customer est entièrement visible dans l'arborescence : controller, dto, mapper, model, repository, service.

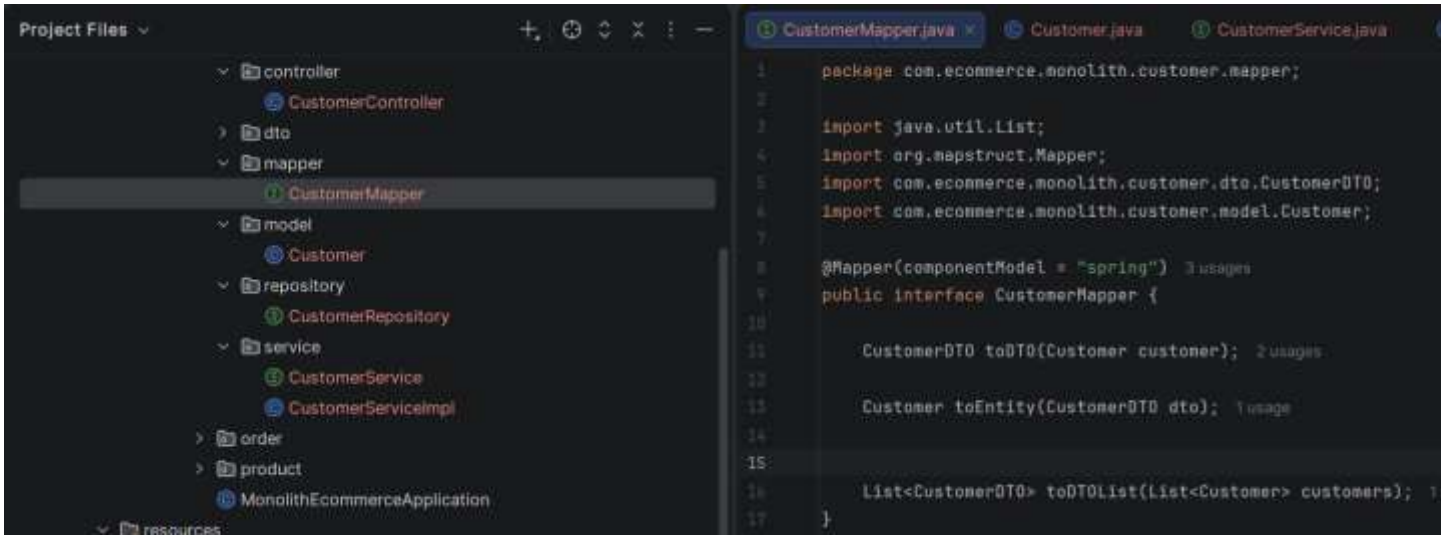

Figure 13 — CustomerMapper.java et arborescence complète du module customer

CreateCustomerRequest est le DTO de requête, contenant uniquement name et email. CustomerDTO est le DTO de réponse avec id, firstName, lastName et email — correspondant exactement aux champs de l'entité Customer exposés à l'API.

Figure 14 — CreateCustomerRequest.java : DTO de requête pour la création de client

Figure 15 — CustomerDTO.java : DTO de réponse avec les 4 champs exposés

5. Tests Postman — Module Customer

Le module Customer est testé via Postman. Deux clients sont créés pour valider le bon fonctionnement des endpoints, puis la liste est consultée pour vérifier la persistance des données.

5.1 POST /api/customers — Création du client 1

Le premier client créé est Thierno Hachimi (email : thierno@gmail.com). Le corps de la requête est un objet JSON conforme au CustomerDTO. La réponse 201 Created renvoie le CustomerDTO avec l'id généré (1) et tous les champs correctement persistés.

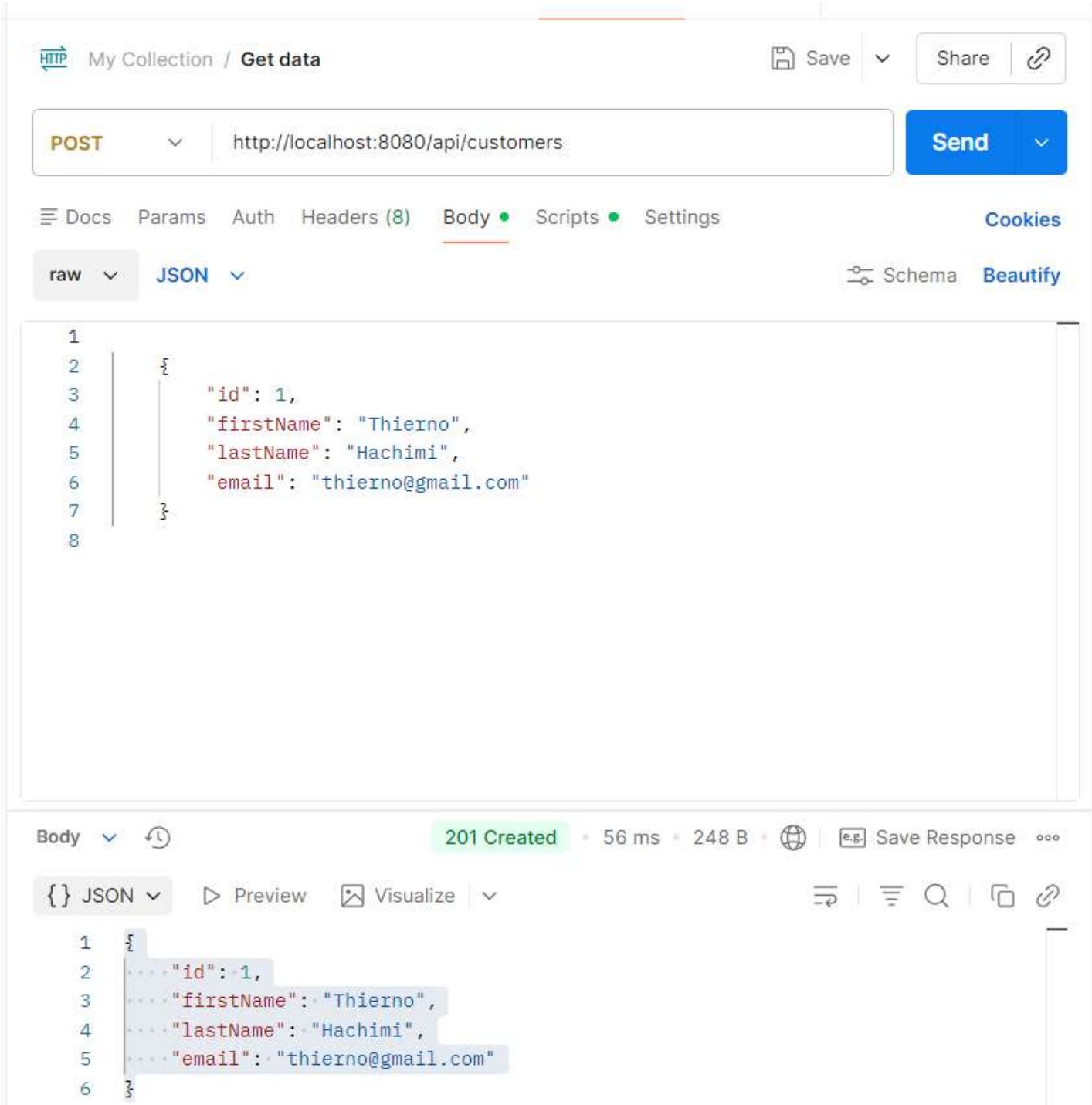

Figure 16 — POST /api/customers : création de Thierno Hachimi (201 Created, id=1)

5.2 POST /api/customers — Création du client 2

Le deuxième client créé est Diallo Lamine (email : hugo@gmail.com). La réponse 201 Created confirme la création avec l'id 2, démontrant l'auto-incrémentation correcte de la clé primaire.

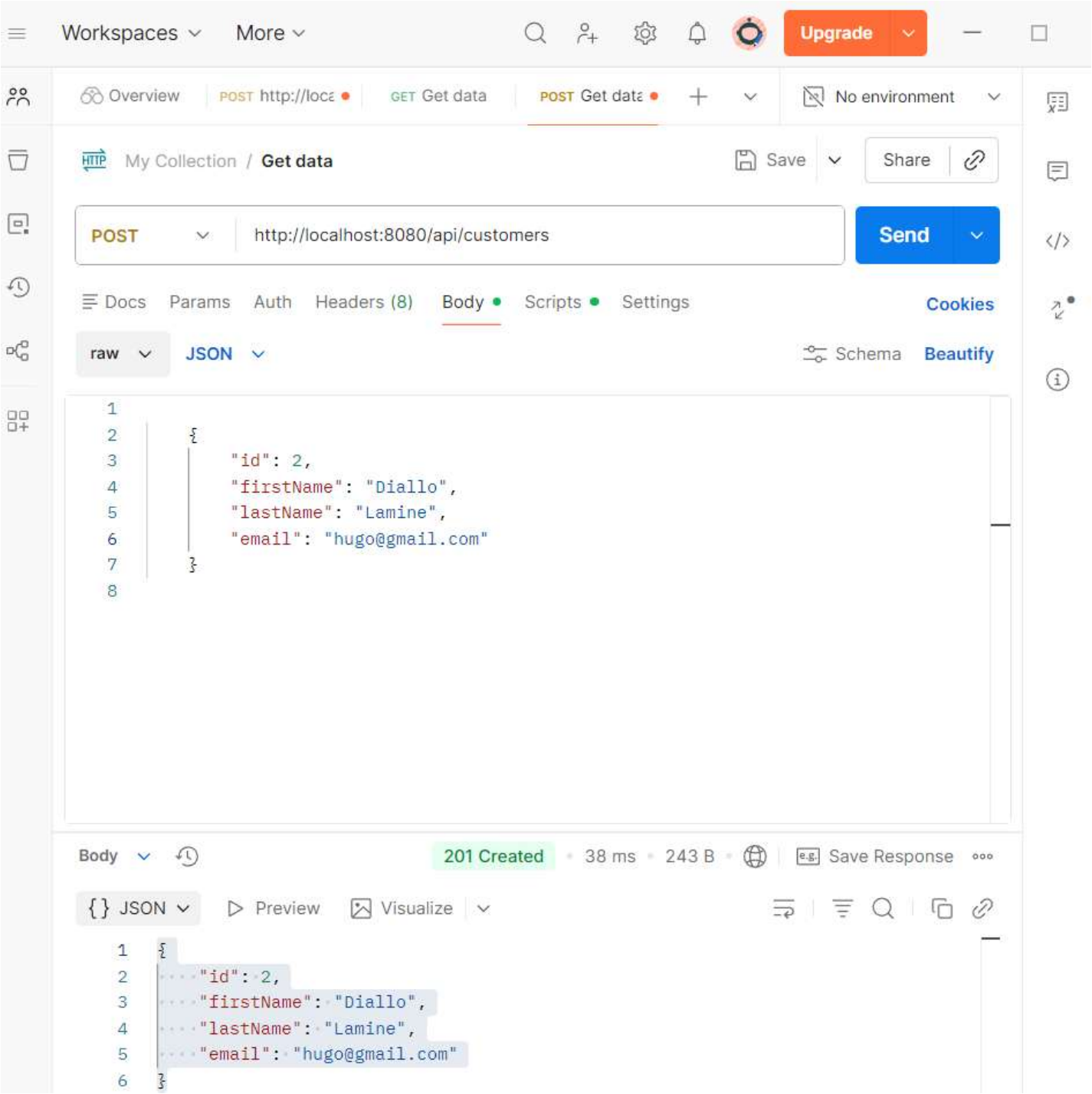

Figure 17 — POST /api/customers : création de Diallo Lamine (201 Created, id=2)

5.3 GET /api/customers — Liste des clients

Un GET sur /api/customers confirme la présence des deux clients en base. La réponse 200 OK retourne un tableau JSON avec les deux CustomerDTO. La structure de la réponse respecte bien le DTO défini : id, firstName, lastName, email.

My Collection / Get data

Save Share

GET http://localhost:8080/api/customers Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Body 200 OK 17 ms 320 B Save Response

JSON Preview Visualize


```
[
    {
        "id": 1,
        "firstName": "Thierno",
        "lastName": "Hachimi",
        "email": "thierno@gmail.com"
    },
    {
        "id": 2,
        "firstName": "Diallo",
        "lastName": "Lamine",
        "email": "hugo@gmail.com"
    }
]
```


Figure 18 — GET /api/customers : liste des 2 clients (200 OK)

6. Module Order

Le module Order est le plus complexe de l'application : il dépend des deux autres modules. Il respecte strictement la règle d'architecture définie dans le TP : Order consomme uniquement les interfaces publiques CustomerService et ProductService, sans jamais accéder à leurs repositories directement.

6.1 Entité Order

L'entité Order est mappée sur la table "orders" — le nom "order" est un mot réservé en SQL, d'où l'utilisation de "orders" pour éviter les erreurs. Elle contient : un id auto-incrémenté, une date de commande (LocalDateTime), un montant total (Double), un statut (PENDING, VALIDATED, SHIPPED) et une relation @ManyToOne vers Customer via la colonne customer_id.

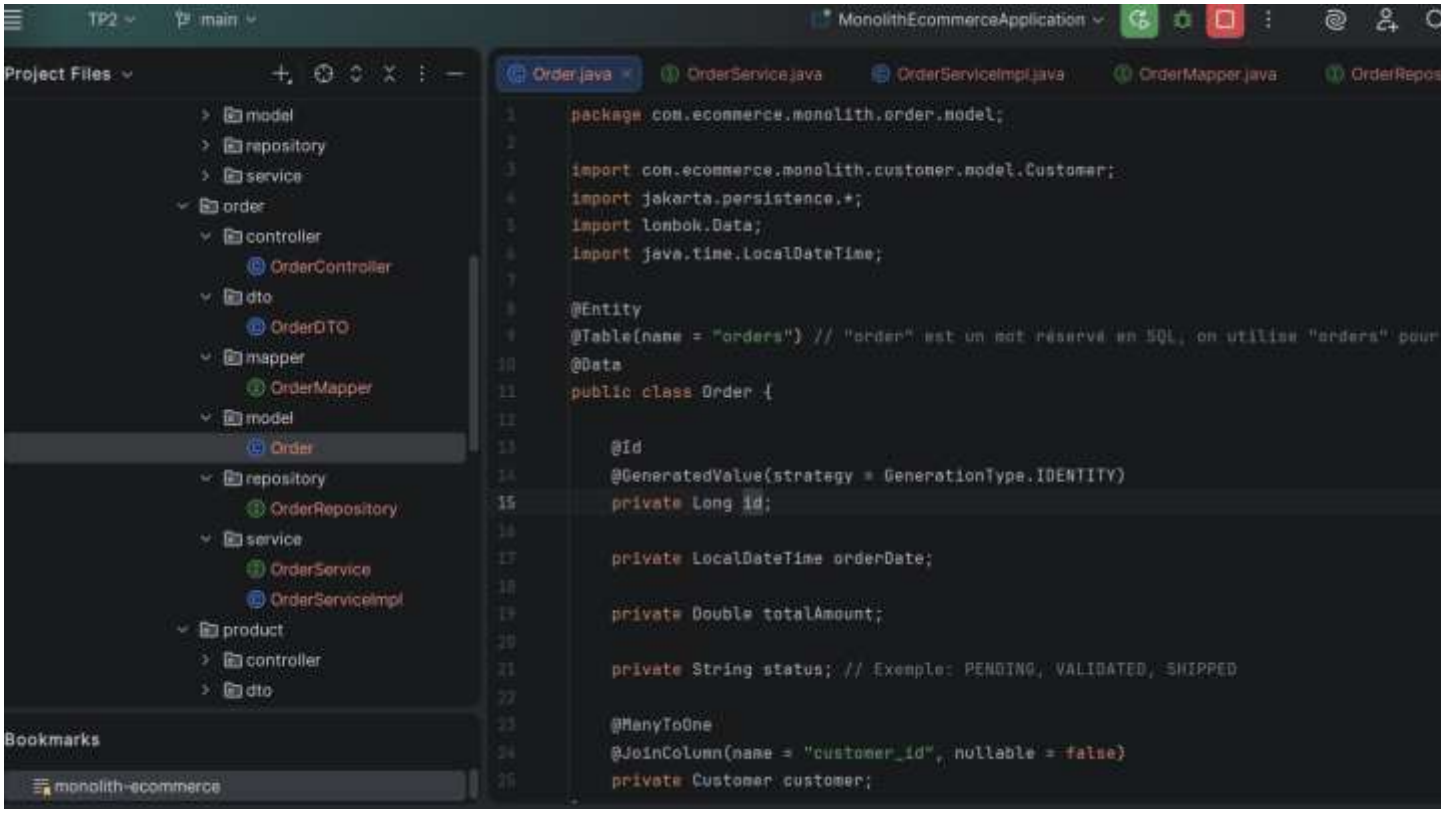

Figure 19 — Order.java : entité avec @ManyToOne vers Customer et arborescence du module order

6.2 OrderMapper

OrderMapper gère la conversion entre Order et OrderDTO. La particularité ici est la gestion de la relation @ManyToOne : @Mapping(source = "customer.id", target = "customerId") extrait l'ID du client depuis l'objet Customer imbriqué. L'inverse @Mapping(source = "customerId", target = "customer.id") reconstitue l'objet Customer depuis l'ID lors de la conversion DTO → Entity.

```
package com.ecommerce.monolith.order.mapper;

import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.model.Order;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(componentModel = "spring")
public interface OrderMapper {

    // On dit à MapStruct : prends l'ID du Customer et mets-le dans customerId
    @Mapping(source = "customer.id", target = "customerId")
    OrderDTO toDTO(Order order);

    // Et l'inverse : prends le customerId et crée un objet Customer avec cet ID
    @Mapping(source = "customerId", target = "customer.id")
    Order toEntity(OrderDTO dto);
}
```

Figure 20 — OrderMapper.java : gestion de la relation ManyToOne avec @Mapping source/target

6.3 OrderService — Interface

L'interface OrderService définit trois méthodes : createOrder(), getAllOrders() et getOrderById(). Cette interface constitue le contrat public du module Order, consommé par le contrôleur.

```
package com.ecommerce.monolith.order.service;

import com.ecommerce.monolith.order.dto.OrderDTO;
import java.util.List;

public interface OrderService {
    OrderDTO createOrder(OrderDTO orderDTO);
    List<OrderDTO> getAllOrders();
    OrderDTO getOrderById(Long id);
}
```

Figure 21 — OrderService.java : interface du module Order

6.4 OrderServiceImpl

L'implémentation de createOrder() suit quatre étapes : (1) vérifier l'existence du client via customerService.existsById(), (2) convertir le DTO en entité via le mapper, (3) initialiser le statut à PENDING et la date, (4) sauvegarder et retourner le DTO. getAllOrders() utilise

Java Streams pour convertir la liste. getOrderById() lève une RuntimeException si la commande est introuvable.

```
public class OrderServiceImpl implements OrderService {

        // 4. Enregistrer et renvoyer le DTO résultat
        Order savedOrder = orderRepository.save(order);
        return orderMapper.toDTO(savedOrder);
    }

    @Override
    public List<OrderDTO> getAllOrders() {
        return orderRepository.findAll().stream()
                .map(orderMapper::toDTO)
                .collect(Collectors.toList());
    }

    @Override
    public OrderDTO getOrderById(Long id) {
        Order order = orderRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Order not found"));
        return orderMapper.toDTO(order);
    }
}
```

Figure 22 — OrderServiceImpl.java : logique de création et récupération des commandes

6.5 OrderDTO

L'OrderDTO expose uniquement l'ID du client (customerId) plutôt que l'objet Customer complet. Ce choix architectural évite la surcharge de données dans la réponse API et respecte le principe de divulgation minimale des informations.

```
package com.ecommerce.monolith.order.dto;

import lombok.Data;
import java.time.LocalDateTime;

@Data
public class OrderDTO {
    private Long id;
    private LocalDateTime orderDate;
    private Double totalAmount;
    private String status;
    private Long customerId; // On ne renvoie que l'ID du client pour rester simple
}
```

Figure 23 — OrderDTO.java : DTO de commande avec customerId (pas l'objet Customer entier)

7. Tests Postman — Module Order

Le module Order est testé en créant une commande liée au client d'ID 1 (Thierno Hachimi), puis en consultant la liste de toutes les commandes.

7.1 POST /api/orders — Création d'une commande

Une commande est créée avec un montant total de 150,50, un statut "pending" et un customerId de 1 (le client Thierno Hachimi). La réponse 201 Created retourne l'OrderDTO complet avec l'id généré (1), la date de commande générée automatiquement au moment de la persistance, et le statut normalisé en majuscules "PENDING".

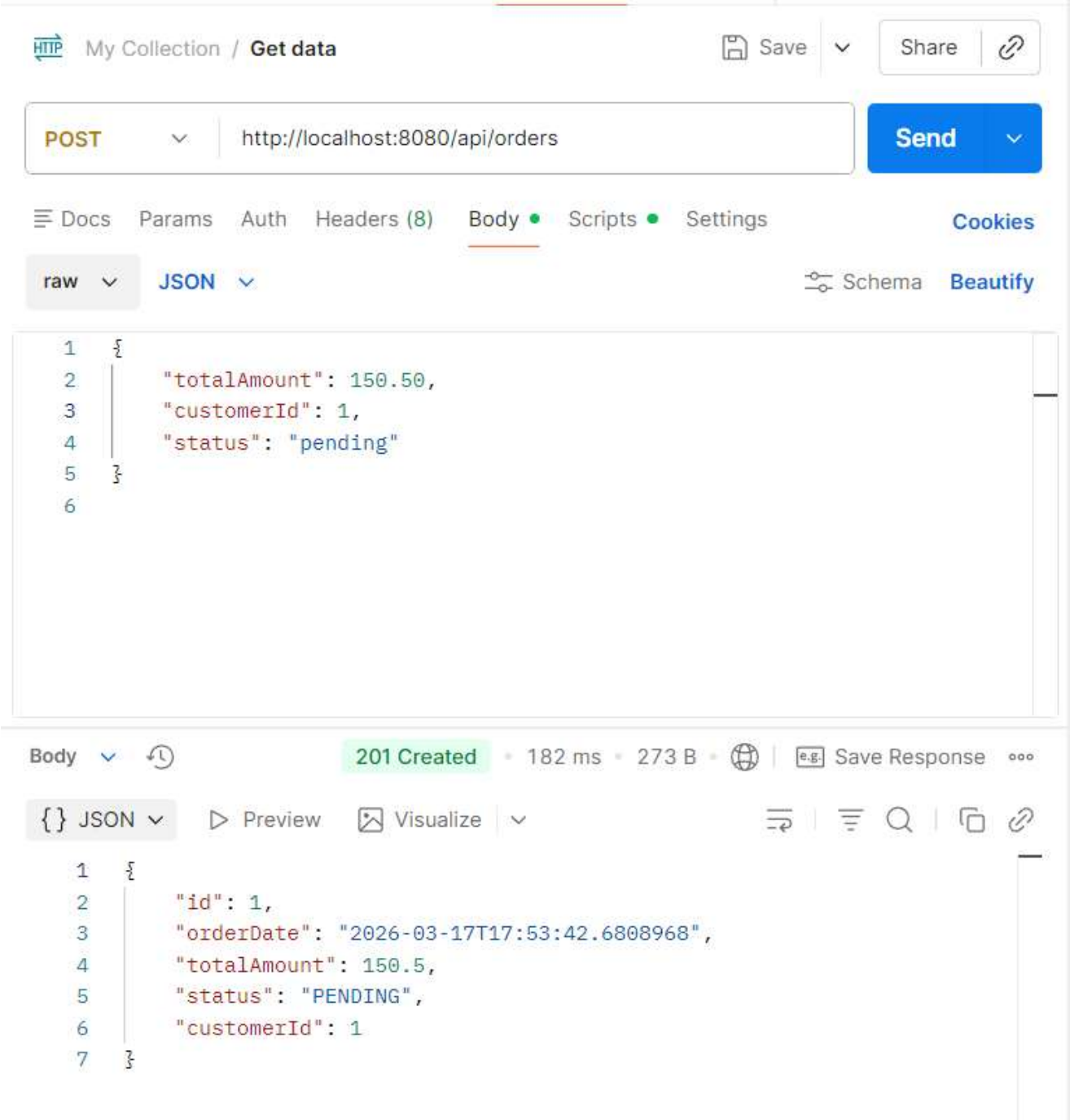

Figure 24 — POST /api/orders : création d'une commande pour le client id=1 (201 Created)

My Collection / Get data

Save Share

POST http://localhost:8080/api/orders Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2     "totalAmount": 150.50,
3     "customerId": 1,
4     "status": "pending"
5 }
6
```

Body 201 Created 182 ms 273 B Save Response

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "orderDate": "2026-03-17T17:53:42.6808968",
4     "totalAmount": 150.5,
5     "status": "PENDING",
6     "customerId": 1
7 }
```

Figure 25 — POST /api/orders : détail de la réponse avec orderDate générée automatiquement

7.2 GET /api/orders — Liste de toutes les commandes

Un GET sur /api/orders retourne la liste de toutes les commandes. La réponse 200 OK confirme la présence de la commande créée avec tous ses champs : id, orderDate, totalAmount, status et customerId. Ce dernier champ illustre le choix architectural de l'OrderDTO qui n'expose que l'identifiant du client.

My Collection / Get data

Save Share

GET http://localhost:8080/api/orders Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Body 200 OK 116 ms 269 B Save Response

JSON Preview Visualize


```
[
    {
        "id": 1,
        "orderDate": "2026-03-17T17:53:42.680897",
        "totalAmount": 150.5,
        "status": "PENDING",
        "customerId": 1
    }
]
```


Figure 26 — GET /api/orders : liste des commandes (200 OK) avec customerId visible

8. Conclusion et Bilan

Ce TP2 a permis de transformer une application monolithique simple en une architecture modulaire bien structurée, prête pour une éventuelle migration vers les microservices. Chaque module est désormais autonome, possède ses propres couches et communique avec les autres uniquement via des interfaces de service.

Compétences acquises

- Conception du pattern DTO pour découpler l'API de la base de données.
- Utilisation de MapStruct pour générer automatiquement le code de mapping à la compilation.
- Séparation service interface / service implémentation pour un code plus maintenable et testable.
- Modélisation d'une architecture modulaire avec des frontières de domaine claires.
- Implémentation d'une relation @ManyToOne entre Order et Customer avec gestion dans le mapper.
- Utilisation du mapper inter-couches pour éviter l'exposition directe des entités JPA.
- Validation de l'existence d'un client avant création d'une commande (existsById).

Avantages du Monolithe Modulaire vs Microservices

Critère	Monolithe Modulaire	Microservices
Déploiement	Simple — un seul artefact	Complexe — N services
Communication	Appels de méthodes Java	HTTP / Messaging (Kafka...)
Transaction	ACID native JPA	Saga pattern nécessaire
Frontières domaines	Claires — interfaces de service	Enforced — réseau
Scalabilité	Limitée — tout ou rien	Granulaire par service
Migration future	Facile — modules isolés	Déjà en microservices

Le monolithe modulaire représente une excellente étape intermédiaire entre le monolithe simple du TP1 et une architecture microservices complète. Il offre la rigueur architecturale des microservices (modules indépendants, interfaces claires, DTOs) tout en conservant la simplicité opérationnelle d'un déploiement unique. Cette approche est idéale pour des équipes en croissance ou des applications de taille moyenne.